UNIVERSIDAD AUTÓNOMA DE LA CIUDAD DE MÉXICO

Licenciatura en Ingeniería de software
Plantel San Lorenzo Tezonco

Técnicas de pruebas de software
Práctica 1. Particiones de Equivalencia con Pytest

Presenta
Edson Joel Carrera Ávila

INTRODUCCIÓN

El presente trabajo aborda la implementación de estrategias de *Caja Negra* para el aseguramiento de la calidad de software, enfocándose específicamente en la técnica de *Particiones de Equivalencia*. El objetivo central es diseñar un conjunto de pruebas optimizado que permita validar la integridad de la lógica de negocio sin necesidad de inspeccionar el código fuente interno.

Para el desarrollo de esta práctica estamos usando el lenguaje de programación *Python*, aprovechando su legibilidad y potencia, junto con el framework *Pytest*. Esta combinación nos permite estructurar los casos de prueba de manera modular y escalable. Mediante el uso de funciones parametrizadas en Pytest, se ha logrado ejecutar múltiples vectores de prueba —tanto de clases de equivalencia válidas como inválidas— sobre las mismas funciones, garantizando una cobertura amplia con un código de prueba mínimo.

A lo largo del repositorio, se detalla el proceso de identificación de las particiones, la definición de los valores límite y la ejecución de los tests automáticos, demostrando cómo una selección inteligente de datos de entrada puede detectar errores críticos de forma eficiente.

DESARROLLO

1. Análisis del Problema 1: clasificar_temperatura

Esta función debe clasificar la temperatura en tres categorías o lanzar excepciones si los valores están fuera de los límites físicos o son de un tipo incorrecto.

Tabla de Particiones de Equivalencia (PE)

Tipo de Partición	Rango / Condición	Clase de Equivalencia	Valor de Prueba	Resultado Esperado
Válida	$-273 \leq T \leq 15$	Temps. frías	10	"Frío"
Válida	$16 \leq T \leq 25$	Temps. templadas	20	"Templado"
Válida	$25 < T \leq 10000$	Temps. calientes	30	"Caliente"
Inválida (Valor)	$T < -273$	Lím. inferior absoluto	-300	ValueError
Inválida (Valor)	$T > 10{,}000$	Lím. superior extremo	12,000	ValueError
Inválida (Tipo)	No numérico	Dato no entero/flotante	"calorcito"	TypeError

Para esta función, es crucial notar que el límite de "Frío" es inclusivo hasta 15 °C, mientras que "Templado" inicia estrictamente en 16 °C. Las particiones inválidas se han definido siguiendo los límites físicos establecidos -273 °C a 10000 °C para asegurar que la función maneje correctamente los errores de entrada o desbordamiento.

2. Análisis del Problema 2: clasificar_calificación

Aquí el sistema evalúa el desempeño académico. A diferencia del anterior, este acepta números decimales.

Tabla de Particiones de Equivalencia (PE)

Tipo de Partición	Rango / Condición	Clase de Equivalencia	Valor de Prueba Sugerido	Resultado Esperado
Válida	$0 \leq C < 6$	Alumno reprobado	5.5	"Reprobado"
Válida	$6 \leq C \leq 8$	Alumno aprobado	7.5	"Aprobado"
Válida	$9 \leq C \leq 10$	Alumno sobresaliente	10	"Sobresaliente"
Inválida (Valor)	$C < 0$	Calificación negativa	-1.0	ValueError
Inválida (Valor)	$C > 10$	Calificación excedida	11.0	ValueError
Inválida (Tipo)	No numérico	Texto o String	"9"	TypeError

En este caso, se han identificado tres particiones válidas principales. Es importante destacar la partición de "Sobresaliente", que solo cubre el rango de 9 a 10. Cualquier valor fuera del rango [0, 10] debe disparar un *ValueError*, lo cual es una prueba crítica para la robustez del método.

IMPLEMENTACIÓN DE FUNCIONES

src/calcular_temperatura.py

```
1 / 1   ... ntos/7mo_TecnicasPruebasSoftware/practicas/practica1/src
1: ing-ejca@LordStaymoon: ~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1/src
  GNU nano 7.2                    calcular_temperatura.py
import sys

def calcular_temperatura(temp):
    # Validación de tipo (TypeError)
    if not isinstance(temp, (int, float)):
        raise TypeError("El valor debe ser numerico")

    # Validación de rango (ValueError)
    if temp < -273 or temp > 10000:
        raise ValueError("Temperatura fuera de rango")

    # Lógica de clasificación
    if temp <= 15:
        return "Frio"
    elif 16 <= temp <= 25:
        return "Templado"
    else:
        return "Caliente"

                                [ 18 líneas leídas ]
^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea
```

src/clasificar_calificacion.py

```
1 / 1   ... ntos/7mo_TecnicasPruebasSoftware/practicas/practica1/src
1: ing-ejca@LordStaymoon: ~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1/src
  GNU nano 7.2                  clasificar_calificacion.py
def clasificar_calificacion(nota):
    # 1. Validación de Tipo (TypeError)
    if not isinstance(nota, (int, float)):
        raise TypeError("La calificación debe ser un numero")

    # 2. Validación de Rango (ValueError)
    if nota < 0 or nota > 10:
        raise ValueError("La nota debe estar entre 0 y 10")

    # 3. Lógica de negocio
    if nota < 6:
        return "Reprobado"
    elif 6 <= nota <= 8:
        return "Aprobado"
    elif 9 <= nota <= 10:
        return "Sobresaliente"

^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea
```

IMPLEMENTACIÓN DE FUNCIONES

tests/test_calcular_temperatura.py

```
1 / 1   ... os/7mo_TecnicasPruebasSoftware/practicas/practica1/tests
1: ing-ejca@LordStaymoon: ~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1/tests
  GNU nano 7.2                test_calcular_temperatura.py
import pytest

from src.calcular_temperatura import calcular_temperatura

@pytest.mark.parametrize("temp, esperado", [
    (10, "Frio"),       # Partición: Frío
    (20, "Templado"),   # Partición: Templado
    (30, "Caliente")    # Partición: Caliente
])
def test_clasificar_temperatura_validas(temp, esperado):
    assert calcular_temperatura(temp) == esperado

def test_error_rango_temperatura():
    with pytest.raises(ValueError):
        calcular_temperatura(12000)

def test_error_tipo_temperatura():
    with pytest.raises(TypeError):
        calcular_temperatura("calorcito")


^G Ayuda     ^O Guardar   ^W Buscar    ^K Cortar    ^T Ejecutar  ^C Ubicación
^X Salir     ^R Leer fich.^\ Reemplazar^U Pegar     ^J Justificar^/ Ir a línea
```

tests/test_calcular_temperatura.py

```
1 / 1   ... os/7mo_TecnicasPruebasSoftware/practicas/practica1/tests
1: ing-ejca@LordStaymoon: ~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1/tests
  GNU nano 7.2                test_clasificar_calificacion.py
import pytest
from src.clasificar_calificacion import clasificar_calificacion

@pytest.mark.parametrize("nota, esperado", [
    (5.5, "Reprobado"),       # < 6
    (7.5, "Aprobado"),        # 6 - 8
    (9.5, "Sobresaliente")    # 9 - 10
])
def test_calificaciones_validas(nota, esperado):
    assert clasificar_calificacion(nota) == esperado

@pytest.mark.parametrize("valor_invalido, excepcion", [
    (-1, ValueError),     # Fuera de rango inferior
    (11, ValueError),     # Fuera de rango superior
    ("Excelente", TypeError) # No numérico
])
def test_calificaciones_invalidas(valor_invalido, excepcion):
    with pytest.raises(excepcion):
        clasificar_calificacion(valor_invalido)


^G Ayuda     ^O Guardar   ^W Buscar    ^K Cortar    ^T Ejecutar  ^C Ubicación
^X Salir     ^R Leer fich.^\ Reemplazar^U Pegar     ^J Justificar^/ Ir a línea
```

CONCLUSIONES

A través de esta práctica, se comprendió la importancia de las **Particiones de Equivalencia** como una estrategia crítica para optimizar el diseño de casos de prueba. En lugar de realizar pruebas exhaustivas, que serían imposibles en rangos tan amplios como los de la temperatura (-273 C a 10000 C), esta técnica permitió validar el comportamiento del sistema utilizando un número mínimo de valores representativos.

Asimismo, la implementación con **Pytest** demostró la eficiencia de las pruebas automatizadas y la importancia de la precisión en los detalles, como el manejo de excepciones (ValueError y TypeError) y la exactitud en los valores de retorno. El uso de parametrización facilitó una cobertura amplia de las clases de equivalencia, asegurando que el software sea robusto tanto ante datos válidos como ante entradas erróneas.

```
1: ing-ejca@LordStaymoon: ~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1
(TPS) ing-ejca@LordStaymoon:~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1$ python -m pytest -v
============================================================================ test session starts ============================================================================
platform linux -- Python 3.10.20, pytest-9.0.2, pluggy-1.5.0 -- /home/ing-ejca/miniconda3/envs/TPS/bin/python
cachedir: .pytest_cache
rootdir: /home/ing-ejca/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1
collected 11 items

tests/test_calcular_temperatura.py::test_clasificar_temperatura_validas[10-Frio] PASSED                                                                                [  9%]
tests/test_calcular_temperatura.py::test_clasificar_temperatura_validas[20-Templado] PASSED                                                                            [ 18%]
tests/test_calcular_temperatura.py::test_clasificar_temperatura_validas[30-Caliente] PASSED                                                                            [ 27%]
tests/test_calcular_temperatura.py::test_error_rango_temperatura PASSED                                                                                                 [ 36%]
tests/test_calcular_temperatura.py::test_error_tipo_temperatura PASSED                                                                                                  [ 45%]
tests/test_clasificar_calificacion.py::test_calificaciones_validas[5.5-Reprobado] PASSED                                                                               [ 54%]
tests/test_clasificar_calificacion.py::test_calificaciones_validas[7.5-Aprobado] PASSED                                                                                [ 63%]
tests/test_clasificar_calificacion.py::test_calificaciones_validas[9.5-Sobresaliente] PASSED                                                                           [ 72%]
tests/test_clasificar_calificacion.py::test_calificaciones_invalidas[-1-ValueError] PASSED                                                                             [ 81%]
tests/test_clasificar_calificacion.py::test_calificaciones_invalidas[11-ValueError] PASSED                                                                             [ 90%]
tests/test_clasificar_calificacion.py::test_calificaciones_invalidas[Excelente-TypeError] PASSED                                                                       [100%]

============================================================================= 11 passed in 0.02s ==============================================================================
(TPS) ing-ejca@LordStaymoon:~/Documentos/7mo_TecnicasPruebasSoftware/practicas/practica1$
```

Repositorio de GitHub:
https://github.com/7mo-TecnicasPruebasSoftware/Practica1_ParticionesEquivalencia